

《中英文翻译系统项目系统》详细设计-金俊贤-09023402

1. 引言
2. 项目概述
3. 总体设计
4. 界面设计
5. 单元模块设计
6. 数据库设计
7. 补充设计和说明

1. 引言

1.1 编写目的

编写本设计的目的是为了准确阐述基于Transformer的中英文翻译系统的具体实现思路和方法，即系统的详细架构和实现逻辑，主要包括程序系统的结构以及各层次中每个程序的设计考虑。预期读者为项目全体成员，包括运行维护和测试人员。

1.2 项目背景

- 系统名称：中英文翻译系统
- 任务提出者：东南大学23级专业技能实训
- 开发者：中英文翻译系统项目组
- 用户和运行该程序系统的计算中心：略。

1.3 定义

1.3.1 技术类

1. 吞吐量

“吞吐量”指系统在单位时间内可处理的输入数量。在 GPU 环境下，系统应能支持至少 50 句/秒的并发翻译吞吐量（批量处理）。

2. 延迟

“延迟”指从用户输入到系统输出结果所需的时间。在指定硬件环境下（CPU: Intel i7, GPU: NVIDIA GTX 1080Ti），对于长度小于 50 个字符的句子，单次翻译平均响应时间应低于 500 毫秒。

3. BLEU Score

“BLEU Score”是一种自动评估机器翻译文本与人工参考译文相似度的指标。在 iic/WMT-Chinese-to-English-Machine-Translation-Training-Corpus-2021 数据集上，本系统的 BLEU Score 可达到 42.50，超过当前主流开源基线模型。

1.3.2 业务类

1. 翻译系统

本项目所指的“翻译系统”是一个集成中英双向神经机器翻译模型，并提供图像识别处理、用户交互等功能的 Web 服务，支持中英文双向翻译、批量翻译，以及 API 接口调用。

2. 源语言

“源语言”指用户输入待翻译文本的原始语言。例如：在中文→英文翻译中，源语言为中文。

3. 目标语言

“目标语言”指用户希望将源文本翻译成的语言。例如：在中文→英文翻译中，目标语言为英文。

4. 输入

本项目所指的“输入”是指用户提交的原始内容，可包括文本、图片、JSON 数据及多格式文档。单次输入不得超过 200 字符。

5. 输出

“输出”是指系统处理输入后返回的翻译内容。系统支持保留输入格式，用户可对结果进行个性化修改。

6. 实时翻译

“实时翻译”指在实时对话或文本输入过程中，系统能够即时完成语言转换的功能。

7. 批量处理

“批量处理”指系统支持用户一次性上传包含大量文本的文件进行翻译，支持多种格式，单个文件大小不得超过 5MB。翻译结果将保持原文件的排版格式。

1.4 参考资料

《中英文翻译系统需求说明》

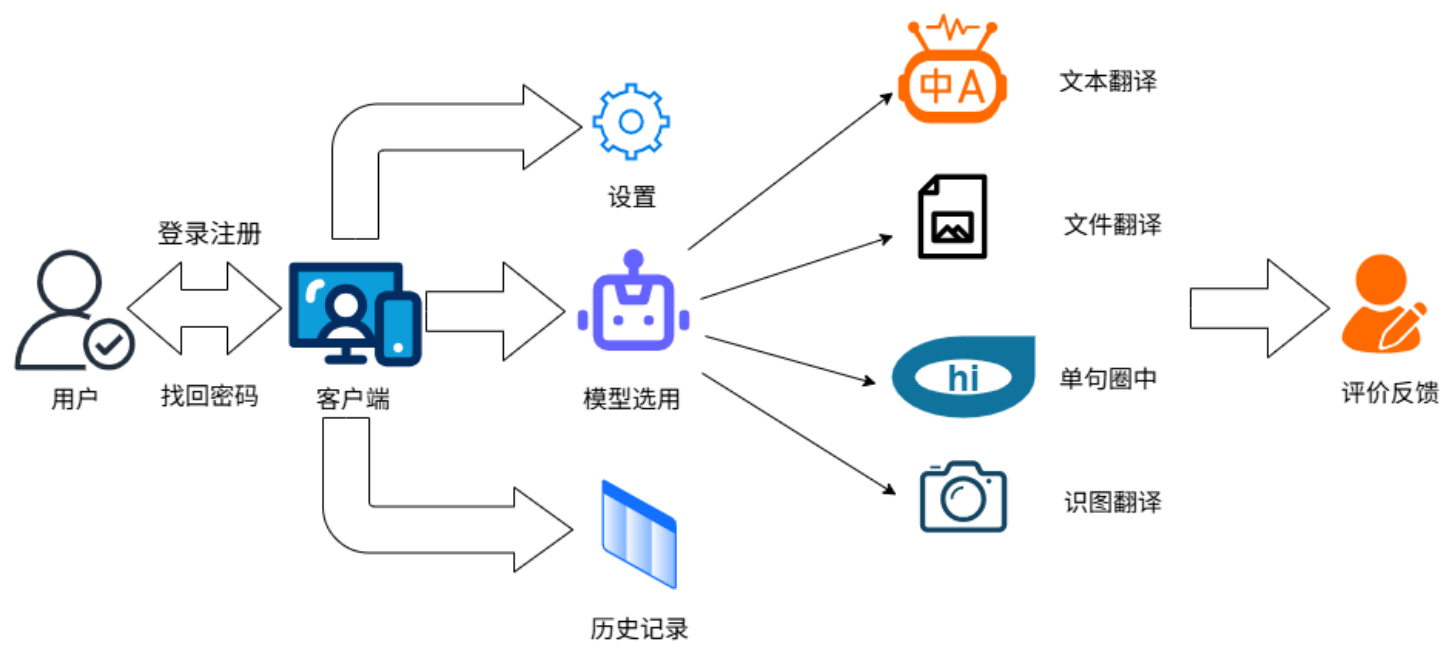
《中英文翻译系统概要设计》

《Python语言编码规范》

2. 项目概述

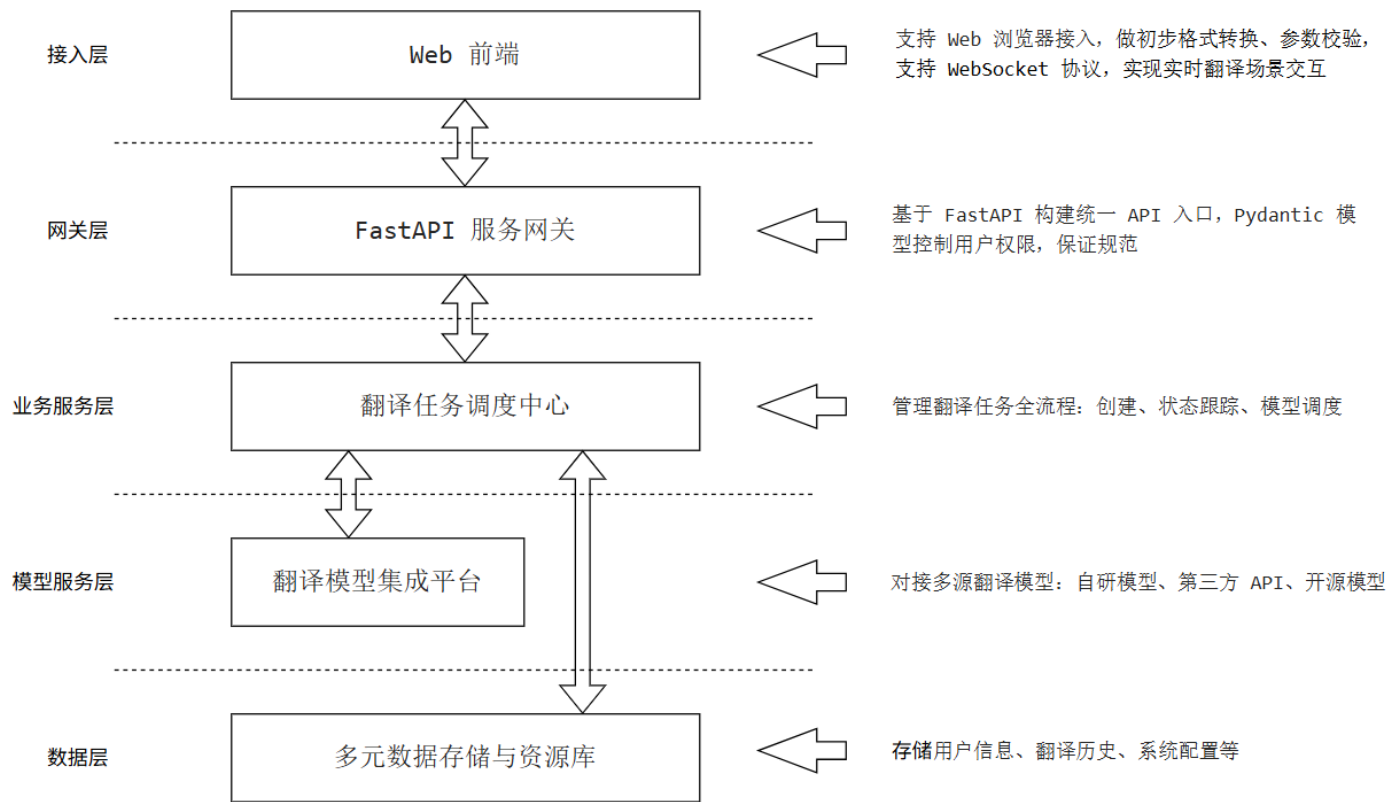
随着跨境交流和全球化进程的加快，企业和个人对高效、准确的翻译工具需求日益增长。无论是在国际商务谈判中，还是在科研资料查询、出国旅行、外语学习等场景下，都需要便捷的中英文翻译支持。为了满足用户在多样化场景下的即时翻译需求，我们自主研发了中英文翻译软件。软件通过统一的语言处理平台，不仅实现了文本、图片和文档翻译，以及翻译历史的高效管理，还提供了模型选用、词汇解读和语音朗读等扩展功能，从而为用户打造一体化的智能翻译解决方案。该软件不仅支持常见的中英文互译，还能为用户提供快速、稳定且高质量的翻译体验，大幅提升跨语言沟通的效率。

结合项目整体目标，系统的功能设计主要分为登陆注册和翻译服务两大部分。登陆注册包含三个主要业务审核：用户注册、用户登录以及找回密码。翻译服务包含八个主要业务审核：文本翻译、识图翻译、文件翻译、单句圈中、历史记录、设置、模型选用以及反馈。



3. 总体设计

3.1 技术架构设计



3.1.1 分层架构

1. 接入层：该层通过 Web 前端接入系统，负责对用户请求进行格式转换和初步校验，同时支持 WebSocket 实时翻译场景，保证前端与后端的高效交互。
2. API 网关层：基于 FastAPI 实现，提供统一的翻译接口入口，支持文本、文档和语音输入。该层完成 API 密钥验证和用户权限控制，使用 Pydantic 模型对请求和响应进行严格校验。
3. 业务服务层：负责翻译任务管理，包括任务创建、状态跟踪和优先级调度。模型调度服务根据翻译类型和语言对选择最优模型，预处理服务进行文本清洗、格式转换和分词处理，后处理服务则优化翻译结果、对齐术语并恢复格式，以提升翻译质量和一致性。
4. 模型服务层：集成自研、第三方 API 和开源模型，提供翻译推理服务，支持同步和异步计算。模型缓存用于存储热门翻译结果和模型参数，性能监控模块记录响应时间和翻译质量指标，便于系统优化和维护。
5. 数据层：包括结构化存储（用户数据、翻译历史、系统配置）、非结构化存储（文档文件、语音数据）以及语言资源库（术语库、翻译记忆库、训练语料）。

3.1.2 控制流程

1. 请求接入与初步处理
 - 用户通过 Web 前端（接入层）发起翻译请求（文本、文档或语音）。
 - 接入层对请求数据进行格式转换（如序列化/反序列化）和初步校验（如非空检查、文件类型检查）。对于需要实时交互的场景（如流式文本翻译），该层建立并维护 WebSocket 连接。
2. API 网关路由与安全控制

- 请求被路由至 API 网关层。
- 网关首先执行身份认证（验证API密钥）和授权鉴权（检查用户是否有权使用请求的服务和资源）。
- 认证授权通过后，网关使用 Pydantic 模型对请求参数进行严格的数据校验，确保数据完整性和合法性。校验失败则立即返回错误响应。

3. 业务调度与任务管理

- 校验通过的请求被转发至业务服务层。
- 任务管理服务为此请求创建一个新的翻译任务，并生成唯一任务ID，用于后续的状态跟踪和结果查询。
- 根据请求的翻译类型（如文本翻译、文档翻译、语音翻译）和目标语言对，模型调度服务决定调用哪个或哪几个模型服务层的模型最为合适（例如，选择自研的特定领域模型或性价比最优的第三方API）。

4. 预处理与模型推理

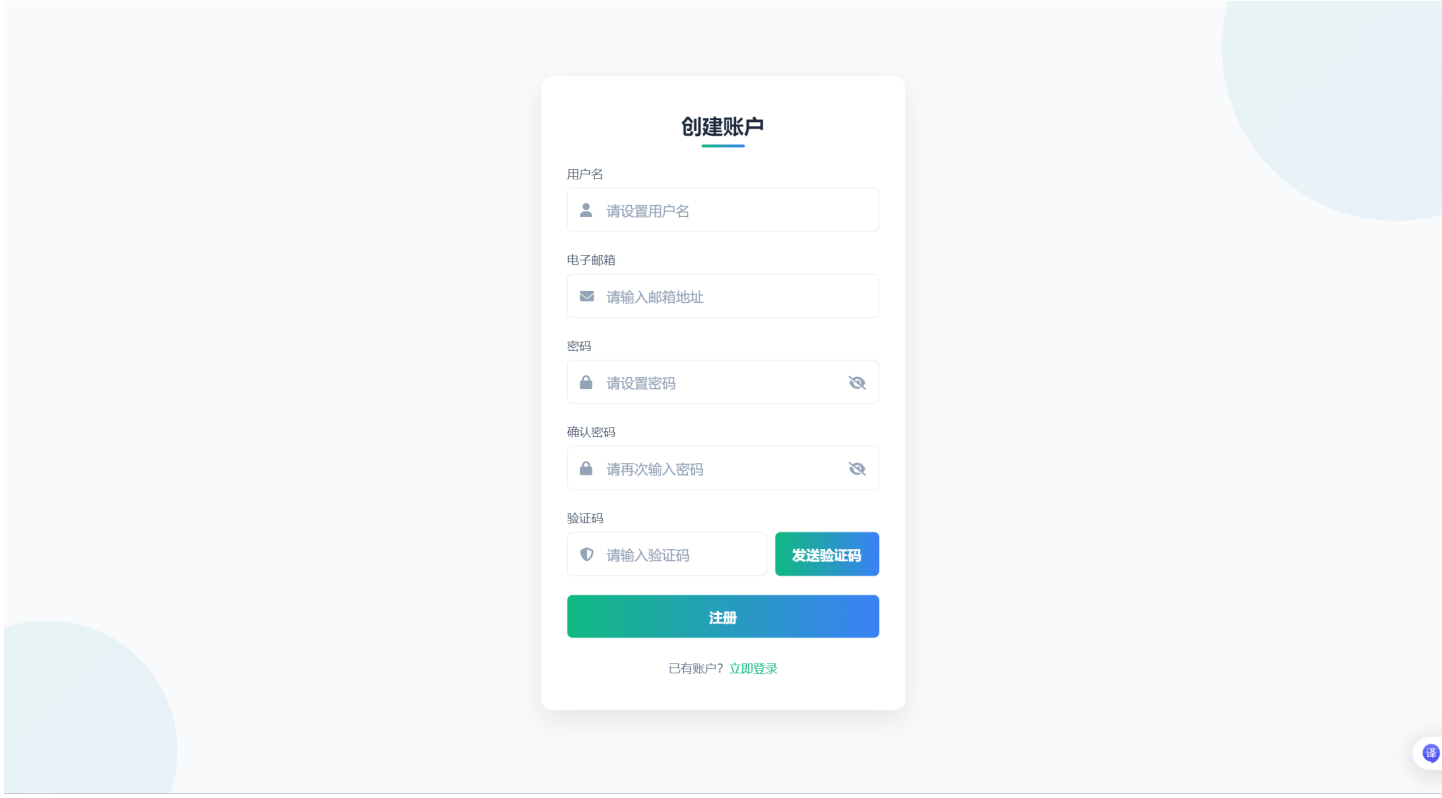
- 预处理服务根据任务类型对输入内容进行处理，例如：文本清洗、语言检测、文档解析提取纯文本、语音识别（ASR）转写等。处理后的标准化文本被送入模型层。
- 模型服务层接收调度指令和预处理后的文本，执行翻译推理。
 - 首先查询模型缓存，检查是否有相同或高度相似的翻译结果可直接返回，以极大降低延迟和计算成本。
 - 若缓存未命中，则调用指定的本地模型、开源模型或第三方API进行计算推理。
- 性能监控模块在此过程中记录模型响应时间、字符数等指标。

5. 后处理与结果优化

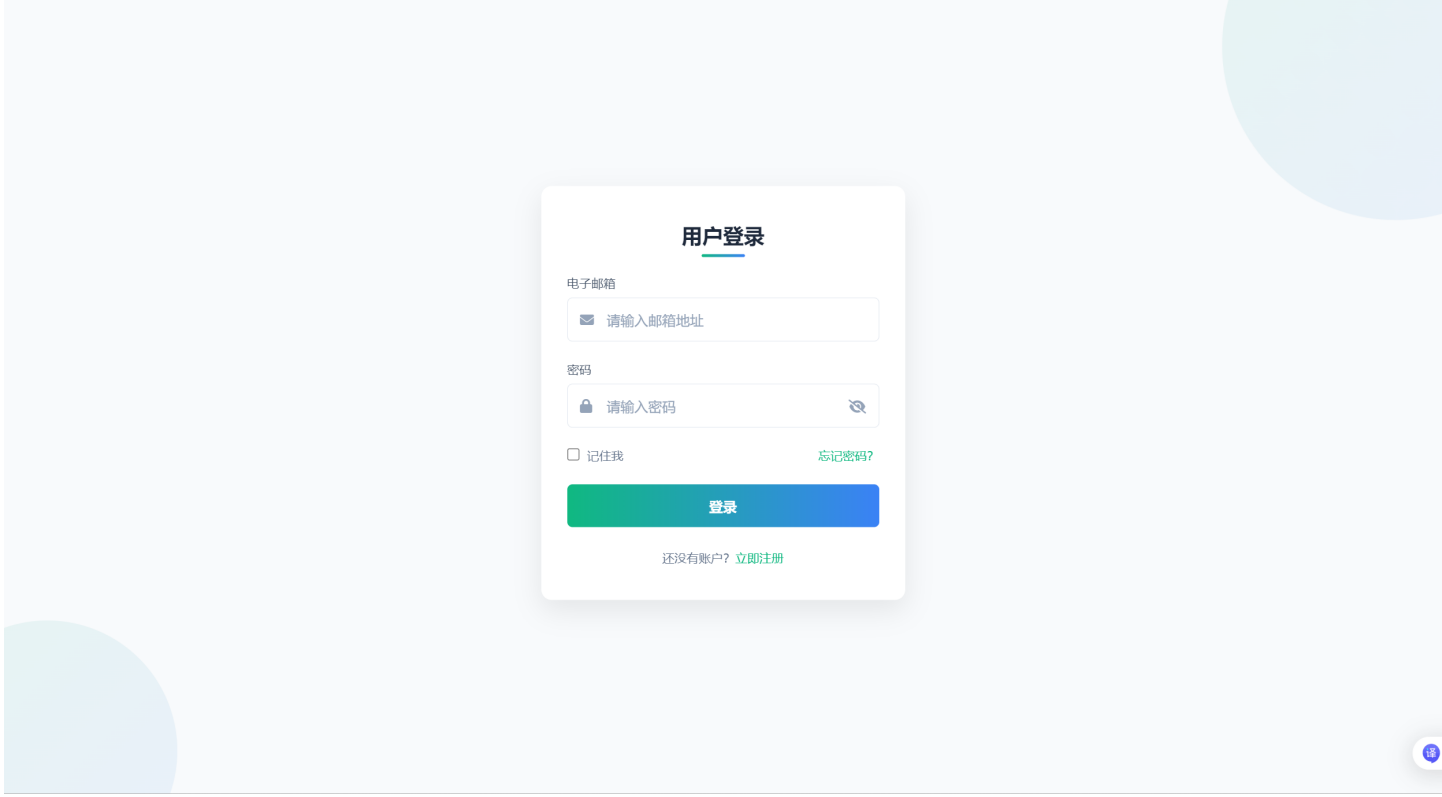
- 获取模型的原始翻译结果后，后处理服务开始工作，执行术语库对齐、翻译记忆库匹配、格式恢复（针对文档翻译）、语言润色等操作，以确保翻译结果符合用户定制要求并保持高质量。

4. 界面设计

4.1 用户注册界面设计



4.2 用户登录界面设计



4.3 找回密码界面设计

找回密码

电子邮箱

✉ 请输入您的注册邮箱

验证码

🛡 请输入验证码

发送验证码

新密码

🔒 请设置新密码



确认新密码

🔒 请再次输入新密码



重置密码

想起密码了? [立即登录](#)

4.4 翻译界面设计

跨语言沟通，无缝连接世界

选择专业翻译模型，获取精准、快速的翻译结果，支持中英互译

选择翻译模型

高速翻译模型

快速响应，适合日常对话翻译

速度优先



高精度翻译模型

专业级翻译，适合文档、学术内容

精度优先



DeepSeek-R1

AI大模型，上下文理解能力强

AI模型



通义千问

支持批量翻译，文化适配性好

大体量



中文

⇌ 英文

请输入要翻译的文本...

0 字符

 选择文件

 单句中选中

 请先选择模型并输入文本或文件



快速响应

毫秒级响应速度，即时获取翻译结果，不等待



精准翻译

AI智能识别语境，确保翻译结果准确传达原意



多语支持

支持全球多种语言互译，满足不同场景需求

文枢

让语言不再是障碍，连接全球沟通

功能

文本翻译
文档翻译
图片翻译

支持

联系我们
使用条款

关注我们



订阅获取最新功能更新

您的邮箱



4.5 历史记录界面设计

翻译历史记录

按原文搜索...

所有记录

由新到旧

返回

导出记录

刷新

时间	原文	译文	操作
2025-09-03 16:00	你好	Hello	☐
2025-09-03 16:05	世界	World	☐
2025-09-03 16:10	图片示例	Picture Example	☐
2025-09-03 16:15	文件示例	File Example	☐

☐ 全选

删除

4.6 设置界面设计

设置

头像上传



上传图片

用户名

老登

保存

字体大小

14px

背景色

浅色

深色

重置

取消

确定

5. 单元模块设计

5.1 数据访问

5.1.1 UserDao

UserDao
+ createUserDao() + retrieveUserDao() + updateUserDao() + deleteUserDao()

1. 成员变量： • userId • email • password	• 用户唯一标识符 • 邮箱账号 • 密码
2. createUserDao(email, password) 3. retrieveUserDao(email) 4. updateUserDao(userId, password) 5. deleteUserDao(userId)	• 创建用户并返回用户唯一标识符(注册) • 查询用户唯一标识符和密码(登录) • 更新用户密码 • 删除用户

5.1.2 SettingDao

SettingDao
+ createSettingDao() + retrieveSettingDao() + updateUsernameDao() + updateAvatarDao() + updateSizeDao() + updateColorDao() + deleteModelDao()

1. 成员变量： • userId • username • avatar • size • color	• 用户唯一标识符 • 用户名 • 头像 • 字号 • 颜色
2. createSettingDao(userId, username, avatar, size, color) 3. retrieveSettingDao(userId) 4. updateUsernameDao(userId, username) 5. updateAvatarDao(userId, avatar)	• 创建用户设置信息 • 查询用户名、头像、字号、颜色(登录) • 更新用户名 • 更新头像

6. updateSizeDao(userId, size)	• 更新字号
7. updateColorDao(userId, color)	• 更新颜色
8. deleteSettingDao(userId)	• 删除用户设置信息

5.1.3 ModelDao

ModelDao
+ createModelDao() + retrieveModelDao() + updateZH_ENDDao() + updateSizeDao() + updateModelsDao() + deleteModelDao()

1. 成员变量： • userId • zh_en • models	• 用户唯一标识符 • 翻译方向 • 模型配置
2. createModelDao(userId, zh_en, models) 3. retrieveModelDao(userId) 4. updateZH_ENDDao(userId, zh_en) 5. updateModelsDao(userId, models) 6. deleteModelDao(userId)	• 创建用户翻译配置信息 • 查询用户翻译配置信息 • 更新翻译方向配置信息 • 更新模型选用配置信息 • 删除用户翻译配置信息

5.1.4 HistoryDao

HistoryDao
+ createHistoryDao() + retrieveHistoryDao() + deleteHistoryDao()

1. 成员变量： • userId • hisId • date • model	• 用户唯一标识符 • 翻译记录对应唯一标识符 • 日期 • 选用模型种类
--	---

• type • input • output	• 翻译文件格式 • 输入文本 • 输出文本
2. createHistoryDao(userId, date, model, type, input, output) 3. retrieveHistoryDao(userId, hisId) 4. deleteHistoryDao(userId)	• 创建翻译记录并返回翻译记录对应唯一标识符 • 查询单条翻译历史信息 • 删除翻译记录

5.1.5 FeedbackDao

FeedbackDao
+ createFeedbackDao()
+ deleteFeedbackDao()

1. 成员变量： • userId • hisId • model • judge • text	• 用户唯一标识符 • 翻译记录对应唯一标识符(已存入HistoryDao) • 反馈模型种类 • 评价好坏 • 评价内容
2. createFeedbackDao(userId, hisId, model, judge, text) 3. deleteFeedbackDao(userId, hisId)	• 创建反馈记录 • 删除反馈记录

5.1.6 AuthTokenDao

TokenDao
- createTokenDao(userId, token, deadline)
- retrieveTokenDao(token)
- updateTokenDao(userId, token, deadline)
- deleteTokenDao(userId)

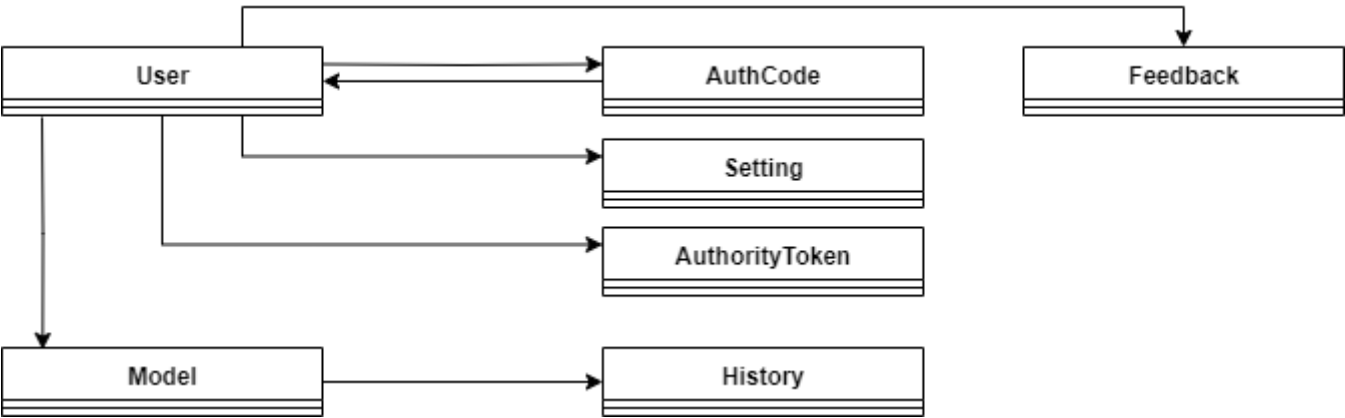
1. 成员变量： • userId • token	• 用户唯一标识符 • 令牌内容
---	---

• deadline	• 截止时间
2. createTokenDao(userId, token, deadline) 3. retrieveTokenDao(token) 4. updateTokenDao(userId, token, deadline) 5. deleteTokenDao(userId)	• 创建令牌信息 • 查询令牌信息(返回userId 和 deadline) • 更新令牌信息 • 删除令牌信息(用户注销)

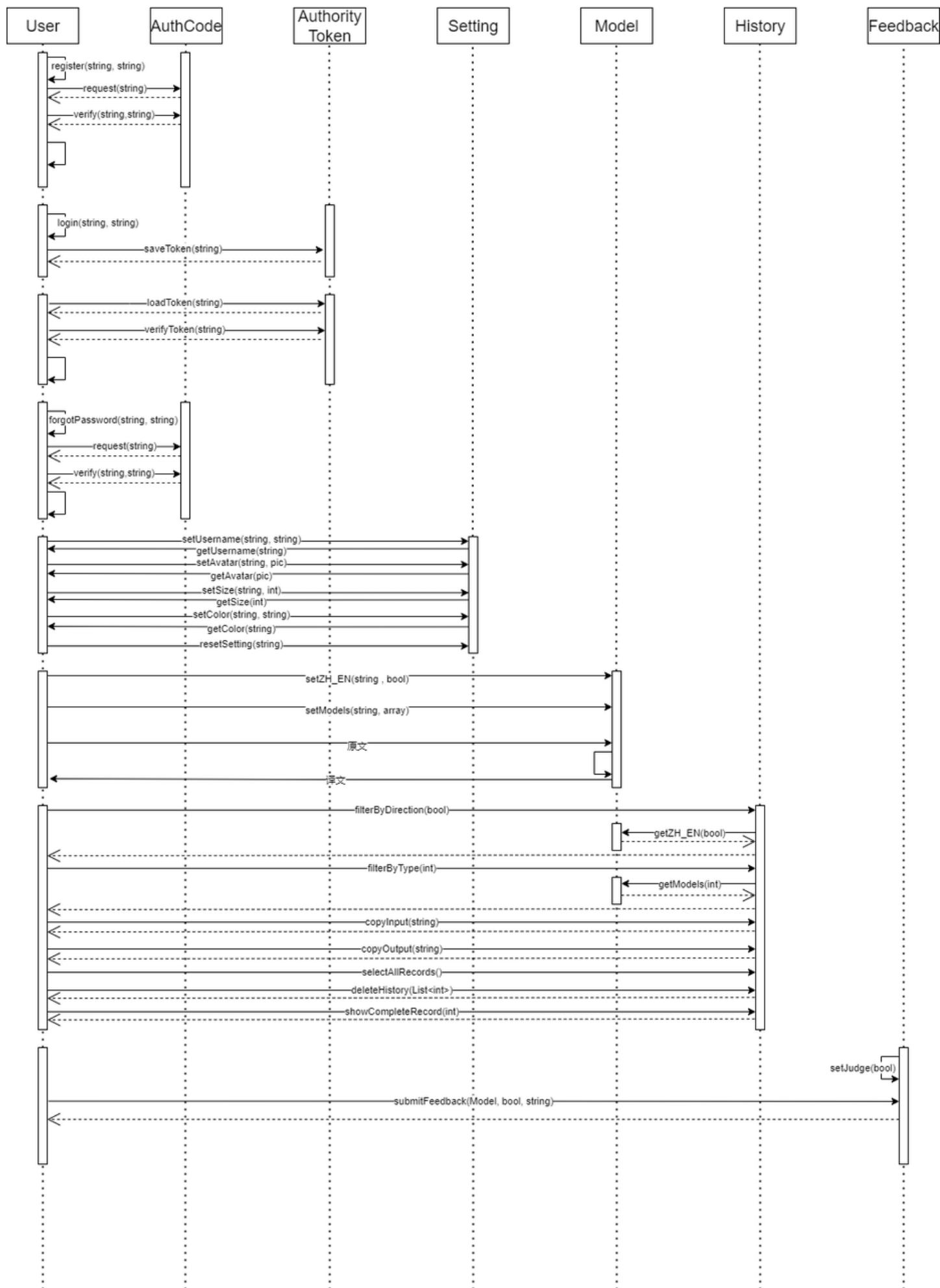
5.2 业务服务

5.2.1 类图设计

5.2.1.1 类关系结构图



5.2.1.2 时序图



5.2.2 类的详细设计描述

5.2.2.1 User

User
-account:string -password:string -username:string -setting:Setting -model:Model
+register() +login() +forgotPassword()

bool register(string account, string password, string username)	用户注册新账户
AuthorityToken login(string account, string password)	用户登录并获取令牌
bool forgotPassword(string account, string newPassword)	用户重置账户密码

5.2.2.2 Setting

Setting
- username: string - avatar: pic - size: int - color: string
+ setUsername(): bool + getUsername(): string + setAvatar(): bool + getAvatar(): pic + setSize(): bool + getSize(): int + setColor(): bool + getColor(): string + resetSetting(): void

bool setUsername(string account, string username)	用户初始化或者更改用户名
string getUsername(string account)	查询用户用户名
bool setAvatar(string account, pic avatar)	用户更改头像
pic getAvatar(string account)	查询用户头像
bool setSize(string account, int size)	用户更改字体大小

int getSize(string account)	查询用户字体大小
bool setColor(string account, string color)	用户更改背景颜色
string getColor(string account)	查询用户背景颜色
void resetSetting(string account)	重置设置信息

5.2.2.3 Model

Model
- zh_en: bool - models: array
+ setZH_EN(): bool + getZH_EN(): bool + setModels(): bool + getModels(): array

bool setZH_EN(string token , bool zh_en)	用户更改翻译方向
bool getZH_EN(string token)	查询用户翻译方向
bool setModels(string token, array models)	用户更改模型配置
array getModels(string token)	查询用户模型配置

5.2.2.4 History

History
- number: int - model: Model - date: string - type: int - input: string - output: string
+ copyInput(string input): string + copyOutput(string output): string + filterByDirection(bool zh_en): List<History> + filterByType(int type): List<History> + showCompleteRecord(int number): void + deleteHistory(List<int> number): void + selectAllRecords(): bool

string copyInput(string input)	复制原文

string copyOutput(string output)	复制译文
List<History> filterByDirection(bool zh_en)	按翻译方向筛选
List<History> filterByType(int type)	按翻译方式筛选
void showCompleteRecord(int number)	在主界面显示完整翻译记录
void deleteHistory(List<int> recordNumbers)	批量删除历史记录
bool selectAllRecords()	全选/取消全选

5.2.2.5 Feedback

Feedback
- number: int - model: Model - judge: bool - text: string
+ ubmitFeedback(Model model, bool judge, string text): bool + setJudge(bool judge): bool

bool submitFeedback(Model model,bool judge, string text)	提交用户反馈
void setJudge(bool judge)	设置反馈类型

5.2.2.6 AuthCode

Authcode
- account: string - postTime: string - code: string
+ request(account): bool + verify(account,code): bool

class Authcode	验证码管理类
bool request(string account)	请求验证码并返回状态
bool verify(string account,string code)	检查验证码是否匹配

5.2.2.7 AuthorityToken

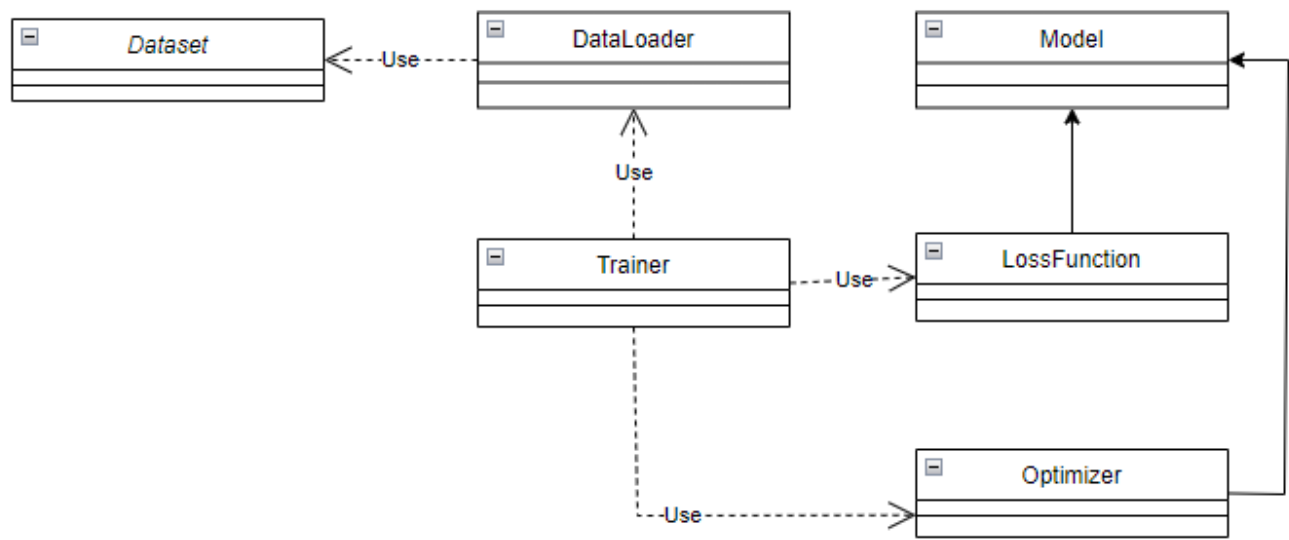
AuthorityToken
- account: string - token: string - deadline: string
+ saveToken(account): void - loadToken(account): bool + verifyToken(account): bool

class AuthorityToken	本地令牌管理类
void saveToken(string account)	生成并保存令牌
bool loadToken(string account)	加载本地令牌
bool verifyToken(string account)	验证令牌

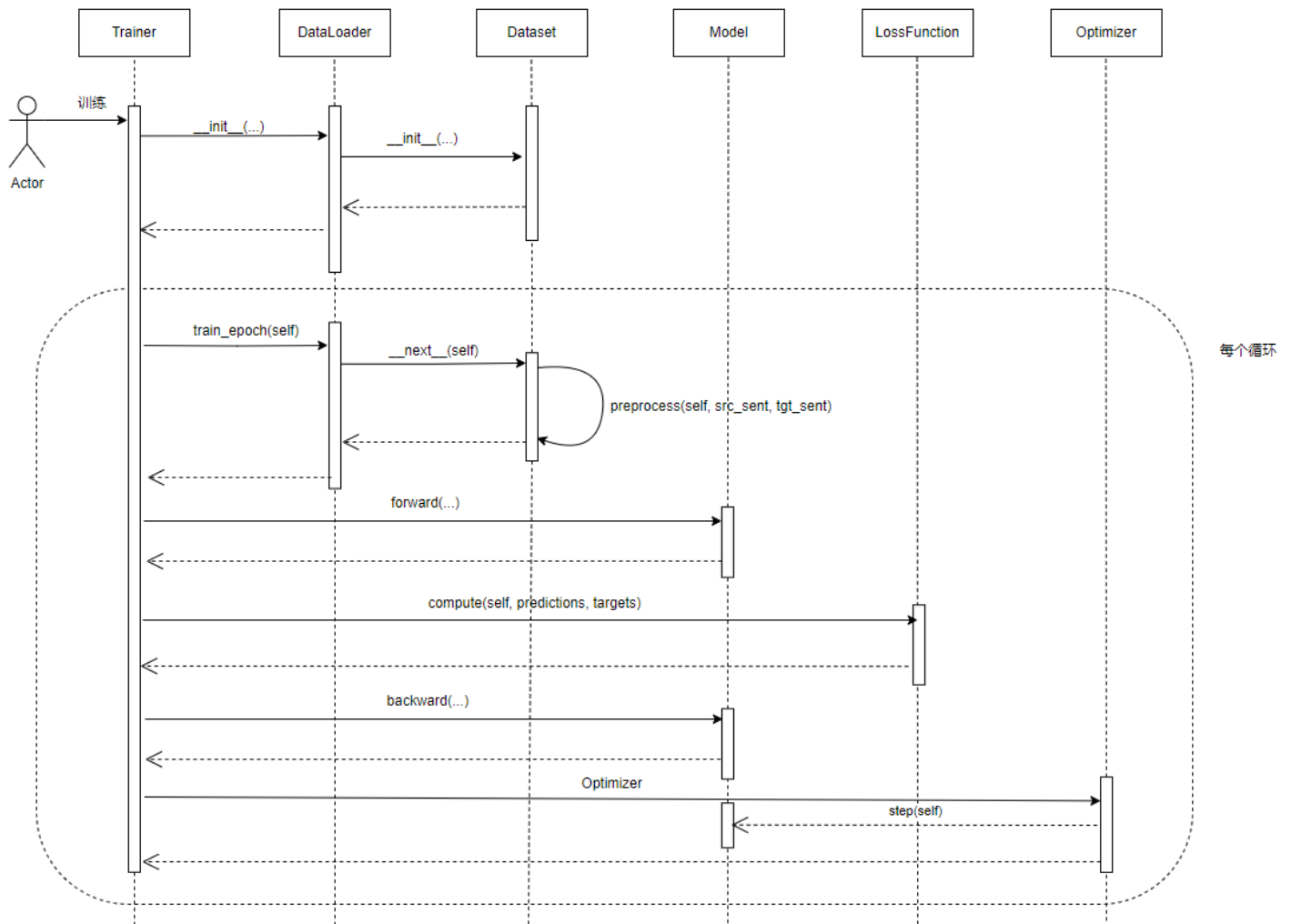
5.3 模型服务

5.3.1 类图设计

5.3.1.1 类关系结构图



5.3.1.2 时序图



5.3.2 类的详细设计描述

5.3.2.1 DataSet

DataSet
- src_file: str - tgt_file: str - src_vocab: dict - tgt_vocab: dict - max_len: int - data: list
+ __init__(self, ...) + __getitem__(self, index) + __len__(self) + preprocess(self, src_sent, tgt_sent)

1. 成员变量：

- | | |
|--|--|
| <ul style="list-style-type: none"> • src_file: str • tgt_file: str • src_vocab: dict • tgt_vocab: dict • max_len: int | <ul style="list-style-type: none"> • 源语言文本文件路径 • 目标语言文本文件路径 • 源语言词汇表（词到索引的映射） • 目标语言词汇表（词到索引的映射） • 序列最大长度 |
|--|--|

data: list	存储处理后的数据样本
2. <code>__init__(self, src_file, tgt_file, src_vocab, tgt_vocab, max_len)</code>	初始化数据集
3. <code>__getitem__(self, index)</code>	获取指定索引的样本，返回(src_tensor, tgt_tensor)
4. <code>__len__(self)</code>	返回数据集大小
5. <code>preprocess(self, src_sent, tgt_sent)</code>	预处理单个句子对

5.3.2.2 DataLoader

DataSet
- dataset: DataSet - batch_size: int - shuffle: bool - batchify_fn: function
+ <code>__init__(self, ...)</code> + <code>__iter__(self)</code> + <code>__next__(self)</code> + <code>_collate_fn(self, batch)</code>

1. 成员变量： - dataset: DataSet - batch_size: int - shuffle: bool - batchify_fn: function	- 数据集实例 - 批量大小 - 是否打乱数据 - 批处理函数（处理填充和打包）
2. <code>__init__(self, dataset, batch_size=32, shuffle=True)</code>	初始化数据加载器
3. <code>__iter__(self)</code>	返回迭代器
4. <code>__next__(self)</code>	获取下一批数据
5. <code>_collate_fn(self, batch)</code>	整理批量数据（填充、转换为张量）

5.3.2.3 Model

Model
+ encoder: Encoder + decoder: Decoder + generator: nn.Linear + src_embed: Embedding + tgt_embed: Embedding + positional_encoding: PositionalEncoding
+ __init__(self,...) + forward(self,...) + encode(self,...) + generate(self,...)

1. 成员变量： - encoder: Encoder - decoder: Decoder - generator: nn.Linear - src_embed: Embedding - tgt_embed: Embedding - positional_encoding: PositionalEncoding	- Transformer编码器 - Transformer译码器 - 线性层，将解码器输出映射到词汇表空间 - 源语言词嵌入层 - 目标语言嵌入层 - 位置编码层
2. __init__(self, src_vocab_size, tgt_vocab_size, d_model, nhead, num_encoder_layers, num_decoder_layers, dim_feedforward, dropout=0.1)	初始化模型
3. forward(self, src, tgt, src_mask=None, tgt_mask=None, memory_mask=None, memory_key_padding_mask=None)	前向传播
4. encode(self, tgt, memory, tgt_mask)	解码目标序列
5. generate(self, memory, max_len, start_symbol)	生成序列

5.3.2.4 LossFunction

LossFunction
- criterion: nn.CrossEntropyLoss - ignore_index: int
+ __init__(self, ignore_index=0) + compute(self, predictions, targets)

1. 成员变量： - criterion: nn.CrossEntropyLoss - ignore_index: int	- 交叉熵损失函数 - 需要忽略的索引（如填充符索引）
--	--

2. <code>__init__(self, ignore_index=0)</code>	初始化损失函数
3. <code>compute(self, predictions, targets)</code>	计算损失值

5.3.2.5 Optimizer

Optimizer
- optimizer: torch.optim.Adam - scheduler: torch.optim.lr_scheduler
+ <code>__init__(self, ...)</code> + <code>step(self)</code> + <code>zero_grad(self)</code> + <code>update_lr(self)</code>

1. 成员变量： - optimizer: torch.optim.Adam - scheduler: torch.optim.lr_scheduler	- Adam优化器实例 - 学习率调度器
2. <code>__init__(self, model, lr=0.001, betas=(0.9, 0.98), eps=1e-9, weight_decay=0)</code>	初始化优化器
3. <code>step(self)</code>	更新模型参数
4. <code>zero_grad(self)</code>	清零梯度
5. <code>update_lr(self)</code>	更新学习率

5.3.2.6 Trainer

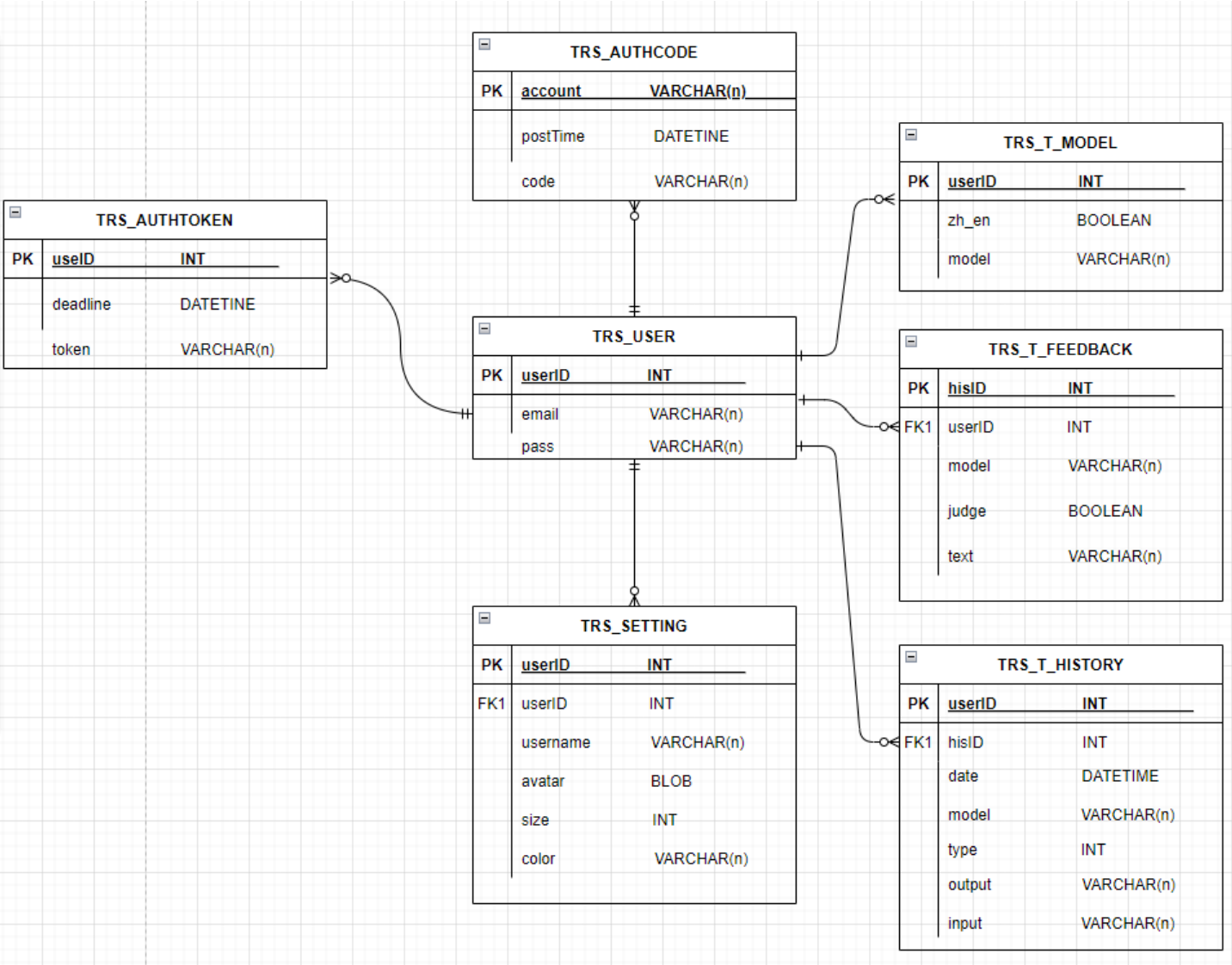
Trainer
-model: Model -loss_fn: LossFunction -Optimizer: Optimizer -train_loader: DataLoader -val_loader: DataLoader -Device: torch.device
+ <code>__init__(self,...)</code> + <code>train(self, epochs)</code> + <code>train_epoch(self)</code> + <code>evaluate(self)</code> + <code>save_model(self, path)</code> + <code>load_model(self, path)</code>

1. 成员变量： - model: Model - loss_fn: LossFunction - Optimizer: Optimizer	- 模型实例 - 损失函数实例 - 优化器实例
--	---

• train_loader: DataLoader • val_loader: DataLoader • Device: torch.device	• 训练数据加载器 • 验证数据加载器 • 训练设备（CPU/GPU）
2. __init__(self, model, loss_fn, optimizer, train_loader, val_loader=None, device='cuda')	初始化训练器
3. train(self, epochs)	训练模型指定轮数
4. train_epoch(self)	训练单个epoch
5. evaluate(self)	在验证集上评估模型
6. save_model(self, path)	保存模型检查点
7. load_model(self, path)	加载模型检查点

6. 数据库设计

6.1 数据库整体结构图



6.2 系统管理

系统管理表格清单

序号	表名	注释
1	TRS_USER	系统用户
2	TRS_SETTING	用户设置
3	TRS_AUTHTOKEN	本地令牌

6.2.1 TRS_USER 表结构

序号	列名	数据类型	注释
1	userId	INT	用户唯一标识符
2	email	VARCHAR(64)	邮箱账号

3	password	VARCHAR(32)	密码
---	----------	-------------	----

6.2.2 TRS_SETTING 表结构

序号	列名	数据类型	注释
1	userId	INT	用户唯一标识符
2	username	VARCHAR(128)	用户名
3	avatar	BLOB	头像
4	size	INT	字号
5	color	VARCHAR(10)	颜色

6.2.3 TRS_AUTHTOKEN 表结构

序号	列名	数据类型	注释
1	account	VARCHAR(64)	账户
2	token	VARCHAR(32)	令牌内容
3	deadline	DATE	截止时间

6.3 业务服务管理

业务服务管理表格清单

序号	表名	注释
1	TRS_T_HISTORY	历史记录
2	TRS_T_FEEDBACK	反馈记录

6.3.1 TRS_T_HISTORY 表结构

序号	列名	数据类型	注释
1	userId	INT	用户唯一标识符
2	hisId	INT	翻译记录对应唯一标识符

3	date	DATETIME	日期
4	type	TINYINT	翻译文件格式
5	input	LONGTEXT	输入文本
6	output	LONGTEXT	输出文本

6.3.2 TRS_T_FEEDBACK 表结构

序号	列名	数据类型	注释
1	userId	INT	用户唯一标识符
2	hisId	INT	翻译记录对应唯一标识符
3	model	VARCHAR(64)	反馈模型种类
4	judge	TINYINT	评价好坏
5	comment	TEXT	评价内容

7. 补充设计和说明

7.1 编译运行环境设计

7.1.1 操作系统与内核

优先选择Ubuntu 22.04 LTS 或 Debian 12（长期支持版本），内核版本≥5.15，确保：

- 对 Python 3.9+、容器化工具（Docker）的原生支持
- 系统库的稳定性，减少依赖冲突
- 长期安全更新，适合生产环境部署

7.1.2 核心工具链

- Python 环境：使用系统内置 Python 3.10+，或通过 pyenv 管理多版本（推荐 3.11+，支持 FastAPI 异步性能优化）。
- 前端工具：安装 Node.js 18+，用于前端资源编译（如 CSS 预处理、JS 打包）。
- 版本控制：git（代码管理）、make（自动化脚本执行）。
- 容器化工具：Docker 与 Docker Compose，用于环境隔离与部署。

7.2 WEB目录结构设计

```
~/Translator/Front-end > master
> tree -lh
[4.0K] .
├── [4.0K] css
│   ├── [6.2K] page-auth.css
│   ├── [8.0K] page-history.css
│   └── [6.0K] page-translate.css
├── [286K] favicon.ico
├── [4.0K] img
│   ├── [ 10K] default_ava.jpg
│   └── [3.8M] logo.jpg
├── [4.0K] js
│   ├── [2.0K] page-feedback.js
│   ├── [6.6K] page-forget.js
│   ├── [ 12K] page-history.js
│   ├── [7.3K] page-login.js
│   ├── [7.2K] page-register.js
│   ├── [ 19K] page-setting.js
│   └── [ 27K] page-translate.js
├── [3.8K] page-forget.html
├── [3.3K] page-history.html
├── [2.7K] page-login.html
├── [4.2K] page-register.html
└── [ 25K] page-translate.html

3 directories, 18 files
```